

CSC 4760 and 5760 Kokkos and MPI Homework (#2)

Anthony Skjellum, Instructor

Assigned: March 27, 2026
Due (in class): April 13, 2026
Submit via Github check-in

Note: (X/Ypts) – X is the points value for CSC 4760, and Y is the points value for CSC 5760.

1 Running problems and examples on TN Tech cluster (5pts/4pts) Please do these steps because they are good for you (enabling for rest of the homework).

1.1 Running Kokkos

To use Kokkos on the cluster, you will want to load the Kokkos packages

```
spack load kokkos@4.7~cuda gcc@14 openmpi cmake
```

You will also want to go to the directory `cd /work/classes/csc4760-001-2026s/<username>` and use your username. Additionally, use either `hpcshell --account=csc4760-001-2026s --cpus-per-task=2` or `salloc --account=csc4760-001-2026s --cpus-per-task=2` to go off the login node onto a node with actual resources. You can change `--cpu-per-task` to be more or less nodes. For more information, use `--help` flag or look up slurm documentation.

From here, you should be able to load and run any Kokkos program given the necessary cmake files. More details on running.

This will work for any of the example or problems.

If you have any issues with running Kokkos, please contact the TAs.

2 Running problems and examples on Delta cluster (5pts/4pts)

Go to <https://openondemand.delta.ncsa.illinois.edu/> and login using Access CI credentials. Note you will have to enroll in a separate Duo MFA and have been given allocation access for the Access CI account. If you login with Access Ci credentials, try getting a new password.

When you have logged in, you will be able to access an HPC OnDemand GUI just like the TN Tech cluster. Compile and run your ring program from the first homework (Problem 1), then screenshot for full credit.

```
[kkruse@dt-login04 hw]$ accounts
Project Summary for User 'kkruse':

Account                Balance(Hours)  Deposited(Hours)  Project
-----
bfrm-delta-gpu         79             100               parallel programmin...
[kkruse@dt-login04 hw]$ salloc --account=bfrm-delta-gpu --partition=gpuA40x4-interactive --nodes=1 --ntasks-per-node=4 --time=01:00:00 --gpus-per-node=1
salloc: Pending job allocation 17433844
salloc: job 17433844 queued and waiting for resources
salloc: job 17433844 has been allocated resources
salloc: Granted job allocation 17433844
salloc: Waiting for resource configuration
salloc: Nodes gpub057 are ready for job
[kkruse@dt-login04 hw]$ mpic++ -o problem1 problem1.cpp
[kkruse@dt-login04 hw]$ ls
problem1  problem1.cpp
[kkruse@dt-login04 hw]$ srun ./problem1
Process 0 recieved integer 0 from process 3
Process 1 recieved integer 0 from process 0
Process 2 recieved integer 0 from process 1
Process 3 recieved integer 0 from process 2
[kkruse@dt-login04 hw]$
```

3 Problems (CSC4760/5760)

3.1 Problem 1 (10pts/8pts):

Given an unsorted array of N integers, sort it in ascending order using a parallel merge sort algorithm with MPI, where the array is evenly distributed across P processes, each process sorts its local chunk, and results are merged back to process 0 through a binary tree reduction.

1

You may implement this as either a regular merge sort performed in each process, or as a general merge sort overall, treating the different processes as parts of the merge sort. Just clarify which option you choose, and make sure that the results are merged back to process 0, through a binary tree reduction. For more information on merge sort, check out <https://www.geeksforgeeks.org/dsa/merge-sort/>.

MERGE SORT IN EACH PROCESS:

```
#include <mpi.h>
#include <iostream>
#include <vector>
#include <algorithm>
#include <time.h>

std::vector<int> merge(const std::vector<int>& left, const std::vector<int>& right){
    std::vector<int> newVector;
    for (int i = 0; i < left.size(); i++){
        newVector.push_back(left[i]);
    }

    for (int i=0; i < right.size(); i++){
        for (int j=0; j < newVector.size(); j++){
            if (right[i] <= newVector[j]){
                newVector.insert(newVector.begin()+j, right[i]);
                break;
            }
        }
        else if(j == newVector.size()-1){
            newVector.push_back(right[i]);
            break;
        }
    }
}

return newVector;
}

int main (int argc, char** argv){
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```

MPI_Comm_size(MPI_COMM_WORLD, &size);

const int N = 16;
int piece = N/size;

std::vector<int> unsort_array(N);
std::vector<int> local_piece(piece);

//create array
if (rank ==0){
    srand(time(NULL));
    std::cout << "Unsorted array: ";
    for (int i=0; i < N; i++){
        unsort_array[i] = rand() %50;
        std::cout << unsort_array[i] << " ";
    }
    std::cout << std::endl;
}

MPI_Scatter(unsort_array.data(), piece, MPI_INT, local_piece.data(), piece, MPI_INT, 0,
MPI_COMM_WORLD);
std::sort(local_piece.begin(), local_piece.end());

int step = 1;
while (step < size){
    if (rank % (2 * step) == 0){
        int partner = rank + step;
        if (partner < size){
            int rcv_size;
            MPI_Recv(&rcv_size, 1, MPI_INT, partner, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            std::vector<int> rcv_data(rcv_size);
            MPI_Recv(rcv_data.data(), rcv_size, MPI_INT, partner, 1, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
            local_piece = merge(local_piece, rcv_data);
        }
    }
    else{
        int partner = rank - step;
        int send_size = local_piece.size();
        MPI_Send(&send_size, 1, MPI_INT, partner, 0, MPI_COMM_WORLD);
        MPI_Send(local_piece.data(), send_size, MPI_INT, partner, 1, MPI_COMM_WORLD);
        break;
    }
    step *=2;
}

if (rank == 0){
    std::cout << "Sorted array: ";
    for (int i=0; i < local_piece.size(); i++){
        std::cout << local_piece[i] << " ";
    }
}

```

```

    }
    std::cout << std::endl;
}

MPI_Finalize();
return 0;
}

```

3.2 Problem 2 (10pts/8pts):

Link and run a program with Kokkos where you initialize a View and print out its name with the `.label()` method.

```

#include <Kokkos_Core.hpp>
#include <iostream>

int main(int argc, char** argv){
    Kokkos::initialize(argc, argv);
    {
        int n = 8;
        Kokkos::View<double*> check("check", n);
        std::cout << check.label() << std::endl;
    }
    Kokkos::finalize();
    return 0;
}

```

3.3 Problem 3 (10pts/8pts):

Make an $n \times m$ View where each index equals $1000 \times i \times j$. Print out the view as well.

```

#include <Kokkos_Core.hpp>
#include <iostream>

int main(int argc, char** argv){
    Kokkos::initialize(argc, argv);
    {
        int n = 10, m = 10;
        Kokkos::View<double**> A("A", n, m);
    }
}

```

```

    for (int i = 0; i < n; i++){
        for (int j = 0; j < m; j++){
            A(i, j) = 1000 * i * j;
        }
    }

    for (int i = 0; i < n; i++){
        std::cout << std::endl;
        for (int j = 0; j < m; j++){
            std::cout << A(i, j) << " ";
        }
        std::cout << std::endl;
    }
}

Kokkos::finalize();

return 0;
}

```

3.4 Problem 4 (10pts/8pts):

Declare a $5 \times 7 \times 12 \times n$ View.

```

#include <Kokkos_Core.hpp>

#include <iostream>

int main(int argc, char** argv){
    Kokkos::initialize(argc, argv);
    {
        int n = 8;

        Kokkos::View<double****> A("A",5,7,12,n);

        std::cout << A.label() << std::endl;
    }

    Kokkos::finalize();
}

```

```
    return 0;
}
```

3.5 Problem 5 (20pts/16pts):

Do simple parallel reduce to output (print) the maximum element in a View. Please do not populate the view with random elements.

```
#include <Kokkos_Core.hpp>

#include <iostream>

int main(int argc, char** argv){
    Kokkos::initialize(argc, argv);
    {
        int n = 8;
        double result;
        Kokkos::View<double*> A("A", n);

        for (int i=0; i < n; i++){
            A(i) = i * 10;
        }

        Kokkos::parallel_reduce("Max Element", n, KOKKOS_LAMBDA(const int& i, double& max){
            if (A(i) > max){
                max = A(i);
            }
        }, Kokkos::Max<double>(result));
        std::cout << result << std::endl;
    }
    Kokkos::finalize();
    return 0;
}
```

3.6 Problem 6 (20pts/16pts):

Create a program that contains both a parallel for loop and a standard for loop for summing rows of a View. Using Kokkos Timer, compare the performance of the two for loops.

```
#include <Kokkos_Core.hpp>

#include <iostream>

int main(int argc, char** argv){

    Kokkos::initialize(argc, argv);

    {

        int n = 100, m = 100;

        double result;

        Kokkos::View<double**> A("A", n, m);

        for (int i=0; i < n; i++){

            for (int j =0; j < m; j++){

                A(i, j) = (i+1) * 10;

            }

        }

        Kokkos::Timer timer;

        for (int i=0; i < n; i++){

            int sum = 0;

            for (int j=0; j < m; j++){

                sum += A(i,j);

            }

            //std::cout << "Row " << i << ": " << sum << std::endl;

        }

        std::cout << "Seconds: " << timer.seconds() << std::endl;

        timer.reset();

        Kokkos::parallel_for ("Row Sum", n, KOKKOS_LAMBDA(const int& i){

            int sum = 0;

            for (int j = 0; j < m; j++){
```

```

        sum += A(i, j);

    }

    //printf("Row %d: %d\n", i, sum);

});

Kokkos::fence();

std::cout << "Parallel time: " << timer.seconds() << " seconds" << std::endl;

}

Kokkos::finalize();

return 0;

}

```

3.7 Problem 7 (20pts/16pts):

Create a program that does adds the elements of a 1D View (a vector) to each row of a 2D View (a matrix) with at least one loop of parallelism.

For a test case: ☐

$A = \begin{bmatrix} \square & \square & 224 & 158 \\ 130 & 158 & 120 & \\ 147 & 115 & 187 & 54 \end{bmatrix}$

$B = \begin{bmatrix} 221 & 12 & 157 \\ 351 & 159 & 344 & 275 \\ 272 & \square & 170 & 277 \\ 445 & 170 & \square & \square \end{bmatrix}$
 $Soln =$
☐

```

#include <Kokkos_Core.hpp>

#include <iostream>

int main(int argc, char** argv){

    Kokkos::initialize(argc, argv);

    {

        int a = 3, b = 3, c = 3;
    }
}

```



```
double result;

Kokkos::View<double**> A("2D", a, b);

Kokkos::View<double*> B("1D", c);


A(0,0)=130; A(0,1)=147; A(0,2)=115;

A(1,0)=224; A(1,1)=158; A(1,2)=187;

A(2,0)=54; A(2,1)=158; A(2,2)=120;


B(0)=221; B(1)=12; B(2)=157;


Kokkos::parallel_for("Row Sum", a, KOKKOS_LAMBDA(const int& i){

    for (int j =0; j < b; j++){

        A(i, j) = A(i,j) + B(j);

    }

});

Kokkos::fence();


for (int i=0; i < a; i++){

    std::cout << std::endl;

    for (int j =0; j < b; j++){
```

```

        std::cout << A(i,j) << " ";

    }

    std::cout << std::endl;

}

}

Kokkos::finalize();

return 0;

}

```

Extra credit (10pts/8pts): make a function and check for correct shape/dimensions.

4 CSC5760—Graduate Student Problems

4.1 Problem 8 (10pts):

Given an unsorted array of N integers on process 0, partition the data into 3 buckets using 2 pivots, distribute each bucket to one MPI process, sort locally with quicksort, and gather the results back to process 0 in sorted order.

For some information on quick sort, check out

<https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/>. 2

4.2 Problem 9 (5pts):

Using the Kokkos parallel scan function, perform a prefix sum on a 1D view (of all 1s). Using the Kokkos timer, measure and print the time taken for the parallel scan, and print the partial sums. Run this several times to compare results between run.

4.3 Problem 10 (5pts):

Do a matrix multiply between a 2D View and a 1D View with at least one loop of parallelism. For full credit, make a function and check for correct shape/dimensions. You can use the Views A and B from Problem 7 on this homework, in order to test your program.

Note: Undergrads may do these problems for extra credit.

